

PROBLEMA 2 (3 puntos)

Se pretende implementar una plataforma de videollamadas que denominaremos *Escaip*. Los usuarios de esta plataforma se caracterizan entre otras cosas por un identificador único. Para que los usuarios accedan a los servicios que se describen a continuación deben identificarse mediante sus credenciales (correo y su contraseña).

Por un lado, los usuarios pueden intercambiar mensajes, caracterizados por un identificador y un texto, con otros usuarios. El sistema debe diferenciar qué usuario manda el mensaje y qué usuario/s recibe/n dicho mensaje.

Por otro lado, los usuarios pueden participar en llamadas y en estas llamadas pueden participar muchos usuarios. De las llamadas se debe conocer su identificador, la fecha en la que se ha realizado y la duración. Los usuarios también pueden participar en videollamadas, que son un tipo especial de llamadas que también se caracterizan por la resolución de vídeo (que puede ser baja, media y alta) y el tráfico que ha generado.

Además, un usuario que está participando en una llamada o en una videollamada puede generar una invitación para que otros usuarios se unan a la llamada. En todo caso, el sistema solo almacenará los usuarios que han participado en la llamada o en la videollamada, sin diferenciar qué usuario ha iniciado la llamada y qué usuario/s la han recibido. Cuando un usuario ha participado en una llamada o en una videollamada, puede valorar la satisfacción con la misma mediante una nota numérica y reportar un error al servicio técnico en el caso que se produzca alguna anomalía.

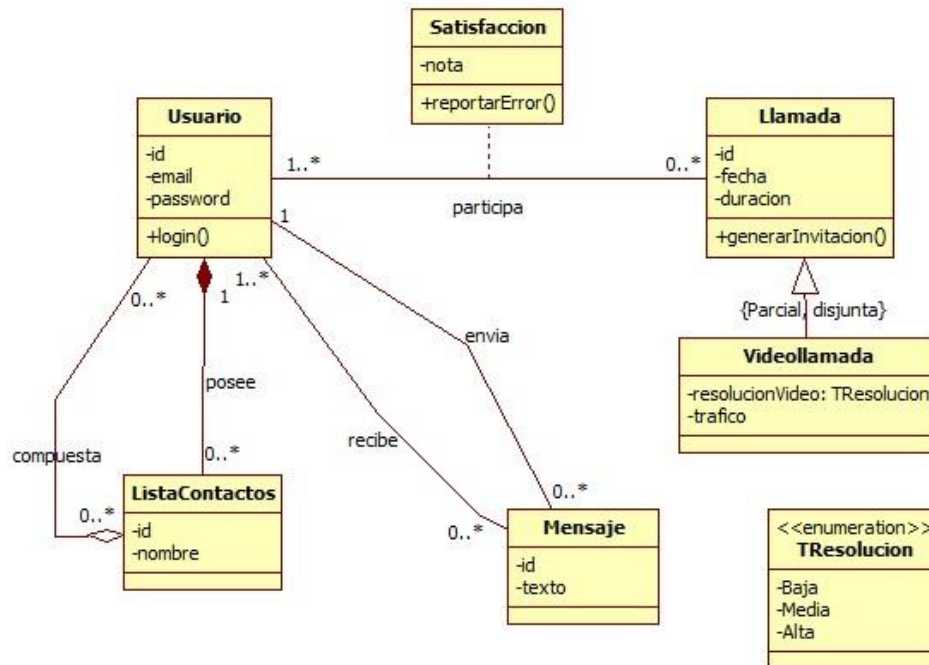
Finalmente, los usuarios pueden tener tantas listas de contactos como deseen. Estas listas se caracterizan por un identificador y un nombre y pueden contener muchos usuarios. Las listas creadas por un usuario se eliminarán cuando este cause baja de la aplicación.

Apartado 2.1 (2,5 puntos)

Realiza un **diagrama de clases de análisis** para modelar el problema descrito. Además, realiza una breve **explicación justificando las decisiones de modelado**.

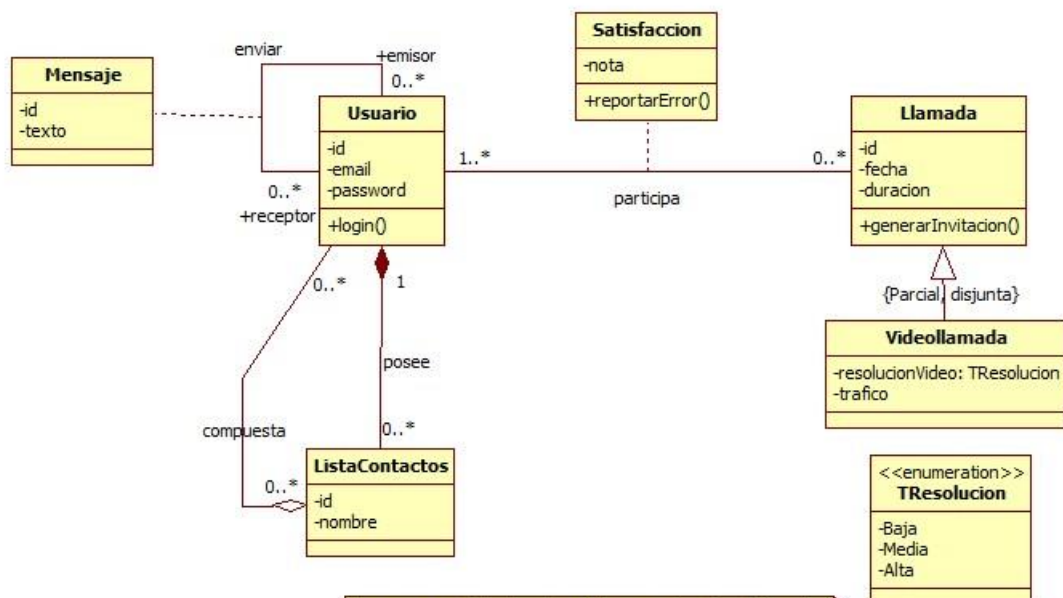
Solución

Opción A



Se asume que todas las clases tienen constructor, destructor, y métodos get y set para cada uno de los atributos

Opción B



Se asume que todas las clases tienen constructor, destructor, y métodos get y set para cada uno de los atributos

Justificación de las decisiones de modelado

Se ha incluido la clase **Usuario**, con un identificador *id* que se indica explícitamente en el enunciado y otros dos atributos que se indican implícitamente (*email* y *password*). Además, la clase contiene el método *login* para la autenticación de usuarios que requiere el sistema. Este método debe ubicarse necesariamente en esta clase porque es la que contiene la información necesaria para llevar a cabo la acción. La clase *Usuario* está relacionada fuertemente con la clase **ListaContactos**, de forma que un usuario puede contener muchos contactos en su lista o ninguno (0..*) y una lista de contactos sólo pertenece a un usuario (1..1). Por ello, existe una relación de agregación inclusiva o composición **posee** con estas cardinalidades entre las clases *Usuario* y *ListaContactos*, ya que la lista de contactos no tiene sentido sin el usuario al que pertenece y tal y como indica el enunciado si un usuario se da de baja, se eliminan sus listas de contactos. Por otro lado, existe una relación de agregación referencial **compuesta** desde *ListaContactos* hacia *Usuario*. En este caso la agregación es referencial porque la eliminación de una *ListaContactos* no implica la eliminación de los *Usuarios* que componen dicha lista. La cardinalidad reflejada indica que una *ListaContactos* puede tener muchos o ningún *Usuario* (0..*) y que un *Usuario* puede estar en muchas o en ninguna *ListaContactos* (0..*).

Se ha incluido la clase **Llamada**, que tiene los atributos indicados en el enunciado (*id*, *fecha*, *duracion*) y un método que permitirá la generación de una invitación para esa llamada. Al igual que antes, este método debe ubicarse necesariamente en esta clase porque es la que contiene la información necesaria para llevar a cabo la acción. También existe la clase **Videollamada**, que hereda de la clase *Llamada*, ya que es un tipo especial de llamada que contiene todas las propiedades de *Llamada* (atributos, métodos y relaciones) y otros atributos específicos para las *Videollamadas*: *trafico* y *calidad*. Se ha modelado *calidad* como un atributo que tiene como tipo de datos *TResolucion*, un tipo enumerado con las tres opciones de calidad que indica el enunciado: *Baja*, *Media* y *Alta*. La relación de herencia es {*Parcial*, *Disjunta*}, parcial porque la clase padre *Llamada* debe ser concreta y disjunta porque, al haber solo una clase hija, no puede haber un objeto que simultáneamente sea instancia de varias clases hijas.

Usuario se relaciona con la clase *Llamada* mediante la asociación **participa**. Fruto de esta relación surge la clase de asociación **Satisfacción**, que almacena la *nota* con la que un *Usuario* valora una *Llamada* y permite reportar un error. Esto no se debería ubicar ni en *Llamada* ni en *Usuario*, ya que la *nota* y el reporte están relacionados tanto con *Usuario* como con *Llamada* y solo se puede dar una *nota* o reportar un error si previamente ha existido una interacción entre objetos de ambas clases. La cardinalidad indicada en *participa* refleja que un *Usuario* puede participar en muchas llamadas o en ninguna (0..*) y que en una *Llamada* debe participar al menos un usuario y pueden participar muchos (1..*).

Por último, se ha incluido la clase **Mensaje**, con el identificador *id* y el *texto* que indica el enunciado. *Usuario* puede relacionarse con *Mensaje* de dos formas. Por un lado, pueden relacionarse mediante dos relaciones de asociación (opción A), **envía** y **recibe** que indican que un *Usuario* puede enviar y recibir ninguno o muchos mensajes (0..*), y que un *Mensaje* debe ser enviado necesariamente por un usuario (1) y puede ser recibido por uno o muchos usuarios (1..*). Por otro lado, estas clases pueden relacionarse mediante una relación reflexiva **enviar** en *Usuario* de la que surge la clase *Mensaje* (opción B). Las cardinalidades y los roles de la relación *enviar* indican que un *Usuario* puede emitir o recibir ninguno o muchos mensajes (0..*). En ambas opciones el sistema puede conocer qué usuario manda un mensaje y qué usuario/s lo recibe/n, tal y como se especifica en el enunciado.

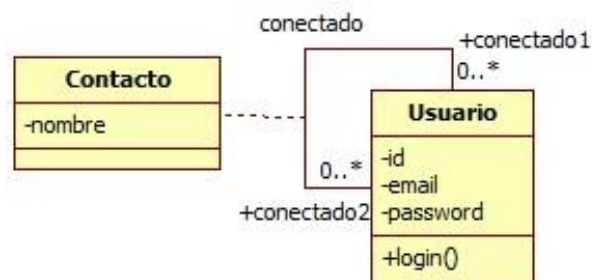
Apartado 2.2 (0,5 puntos)

Suponiendo que en el enunciado del problema se añadiese la siguiente frase:

“Los usuarios pueden estar conectados con otros usuarios, fruto de esta conexión cada usuario debe indicar el nombre o alias con el que desea nombrar al usuario con el que está conectado.”

¿Qué **modificaciones harías en el diagrama de clases**? Indica los cambios representando únicamente la parte del diagrama que se ve afectada por el cambio y **explica brevemente las decisiones de modelado**.

Solución



El almacenamiento del nombre o alias que un usuario le asigna a otro usuario cuando están conectados se realiza a través de la relación reflexiva **conectado** y la clase de asociación **Contacto**. Esta clase incluye el atributo *nombre*, donde se almacena el alias que el usuario *conectado1* le puede asignar al usuario *conectado2* cuando se conectan. Esta clase asociada a la relación reflexiva *conectado* y las cardinalidades indicadas permiten que un *Usuario* pueda asignar un nombre a cada uno de los usuarios con los que esté conectado y que un *Usuario* puede ser nombrado de muchas formas por los distintos usuarios con los que esté conectado.

Nota: Las justificaciones que se aportan en la solución son más extensas de lo requerido en el examen y se aportan con motivos docentes.